

Finding buyers for small businesses

Kunal Srivastava, kunal@masagroup.com, 09/20/2026

Repo: <https://github.com/sfkunal/buyerlist>

I built a tool that takes an SMB's website and returns a ranked shortlist of private equity firms that might buy it, with a thesis for each and a note on why each might not work. One command, a few minutes, about fifty cents, saving hours of associate work.

```
$ buyerlist run https://www.carolinapest.com

Carolina Pest Management – NAICS 561710 – Monroe, NC
EBITDA $1.2M–$4.4M (low confidence)
filtered 170 → 93 candidates

1 Thompson Street Capital via PestCo Holdings tier_1 82
2 Incline Equity Partners via Barefoot Mosquito & Pest tier_1 80
3 Gridiron Capital via Greenix tier_1 76
4 Kian Capital Partners via Diamond Landscaping tier_2 73
5 Alpine Investors via Apex Service Partners tier_2 72

$0.5596 · 231s
```

PestCo, Barefoot and Greenix are all real pest control roll-ups. Ten of the twelve results carried a named platform on a sector I never explicitly tuned for.

The obvious approach doesn't work

The first thing anyone tries is asking a language model directly: *which PE funds would buy this HVAC company in Ohio?* I tried it. It named Blackstone and KKR, neither of which buys \$5M-EBITDA contractors, and then invented a firm that doesn't exist.

That isn't a prompting problem, and no amount of prompt engineering fixes it. Naming mid-market PE firms and their portfolio companies is a *recall* task over thousands of thinly-documented private entities, which is precisely where language models are least reliable and most confident. The answers come back fluent, specific, and wrong in a way that's pretty hard to catch. "Meridian Capital Partners acquires niche manufacturers across the Midwest" reads perfectly well whether or not Meridian exists.

So I stopped asking the model to remember things.

What I built

The design rule is one sentence: **the model is never allowed to name a fund. It can only rank funds retrieved from an index I built.**

This isn't a claim that LLMs are useless here, they do most of the heavy lifting. They read messy brochure HTML into structured records, infer company size from indirect evidence, and score fit against a rubric, all things they're genuinely good at. What they can't do is recall which of thousands of lower-middle-market sponsors owns a pest control roll-up in the Carolinas. The system splits along exactly that line: retrieval is mine, and judgment is the model's.

The index is the asset. I scraped 170 PE firms' own websites for investment criteria, check sizes, geography, stated exclusions, and full portfolios: 1,800 portfolio companies and 379 add-on acquisitions, for about \$2.70. It's committed to the repo, so the tool runs without rebuilding.

Matching happens at the **portfolio-company level, not the fund level**, and that's what makes the output useful. Deals here don't happen because a fund "likes business services." They happen because a fund owns a regional roll-up that's done six tuck-ins and wants a seventh. So the matching unit is the platform and its acquisition history, and the add-on thesis is the highest-weighted scoring dimension at 25%. The difference is between *"Alpine invests in services"* and *"Alpine, via Apex Service Partners, which has been buying founder-owned HVAC businesses in adjacent markets."*

Reliability: making the output trustworthy

The hard part of this problem isn't getting an answer, it's knowing whether to believe one. Five things do that work:

Structured outputs everywhere. Every model call goes through `messages.parse()` against a Pydantic schema, so schema violations get retried at the tool-call layer instead of surfacing as JSON parse errors three functions downstream. No regex, no prose parsing.

The allowlist is enforced in code, not prompted. The ranking prompt forbids naming anything outside the candidate set, which is necessary but not sufficient. So after ranking, the pipeline drops any firm *or platform* name that isn't in the candidate list and reports what it dropped. The rule has an observable failure count rather than being a hopeful instruction. Platform names matter as much as firm names here, since the add-on thesis runs through them.

Every claim carries a quote. Fields in the company profile require verbatim supporting text and a source URL, and a quote that merely sits *near* a fact doesn't count. If the extractor can't find text containing the fact, the field comes back null rather than inferred.

Failures are loud. `stop_reason` is checked on every call: a safety refusal and a `max_tokens` truncation each raise a named error identifying the stage. Truncation used to surface as a silent schema retry, which is the worst possible failure mode for a tool like this. Sizing is reported as

a *range with a confidence level*, never a point estimate, and low confidence widens the downstream filter instead of narrowing it.

A second pass argues against the first. An adversarial call takes the top matches and makes the strongest case *against* each: size mismatch, geographic stretch dressed up as adjacency, a platform that's stopped acquiring. For a banker, "why this might not work" is the most useful column on the page.

The reliability work paid off in a way I didn't expect. My first end-to-end run didn't surface Alpine for an HVAC target, which was obviously wrong. Alpine's portfolio page lists Apex under a generic "Services" category next to a specific "HVAC Plumbing & Electrical" vertical, and my extractor had mapped the generic label - Apex came out coded as janitorial and landscaping. The single most obvious buyer in the index was invisible. I rewrote the prompt so the specific vertical wins, rebuilt, and added an automated check that flags any platform mapped to generic administrative codes while its description names a specific trade. It now runs over the whole index on every build, alongside ten other consistency checks (82% of entries clean).

Effective API Use and Cost: \$2.70 to build the index, ~\$0.56 a run

Send the model less. The biggest lever wasn't model choice, it was input size. Rather than feeding whole sites to the model, I score and select the ~12 highest-value pages per site (criteria, portfolio, about, locations) and cap each page's token budget by its score: a criteria page gets 12K characters, a news listing gets 2.5K. That cut a typical fund from ~15K tokens to ~3.5K, roughly a 4× saving before any other decision, and Alpine's portfolio table survived intact.

Match the model to the work. Extracting criteria from an explicit "investment criteria" page is mechanical and doesn't need a frontier model. Haiku handles the index at \$0.023 per firm; Sonnet handles messier SMB sites; Opus is reserved for size inference, ranking, and the adversarial pass. Model-specific parameters are gated by family, since Haiku errors on `effort` rather than ignoring it.

Cache the stable prefix. The fund index sits in the ranking call's system prompt behind a `cache_control` breakpoint. It's byte-identical on every run, so after the first call it bills at roughly a tenth of input rate. Getting the multiplier right mattered: I initially priced the write at 1.25× when the 1-hour TTL I actually request bills at 2×, which understated the headline cost in the one place the cost claim lives.

Don't call the model when Python will do. Size, sector, geography and stated-exclusion filters are pure Python and cut 170 funds to a few dozen before any ranking call. There's no vector database either. At this scale, exhaustively scoring the survivors in one cached call beats approximate retrieval and removes an embedding dependency entirely.

Measure it rather than assert it. Every run prints tokens, cache hits and dollars by stage. An unknown model raises instead of silently pricing at zero.

Does it work?

I backtested against 54 real acquisitions from 2024–26, each traced to an actual announcement. Before scoring a deal, a leakage guard rebuilds the index as it stood *before* that deal, removing the target from any firm's portfolio, dropping add-ons announced later. It stripped 2,182 entries across the run. That mattered: one target was sitting in its acquirer's own portfolio page, which would otherwise have been a free win.

On the 18 deals whose acquirer was in the index, against 170 funds:

recall@1	recall@5	recall@10	median rank when found
22%	39%	44%	1.5

Four deals put the actual acquirer first. Ten of eighteen were misses.

The more useful number is underneath: **only 21 of 54 real acquirers existed in my index at all.** Ranking quality and index breadth are different failures with different fixes, and a single blended figure would have sent me tuning the scoring rubric when the evidence points squarely at coverage. Separately, recall@10 and recall@20 are identical. Nothing was found between ranks 11 and 20, which says the misses are hard-filter losses, not the ranker burying something it saw.

Limitations

The index is too small. 170 funds covers 39% of the acquirers in my test set. This is the binding constraint on the whole system and the first thing I'd fix.

The backtest measures a proxy, and n is small. Eighteen deals puts roughly ± 0.23 around recall@10, so treat it as directional. More importantly, "did we find *the* acquirer" is the wrong target: a buyer list is trying to produce ten credible buyers, not predict the winner. If the tool surfaces ten plausible funds and the business sells to the eleventh on price or chemistry, recall records a miss on a list that was fine. 44% is a floor, not a grade. The eval set also skews toward targets whose sites survived acquisition, since absorbed companies redirect to the buyer.

What I'd do next

Replace the curated seed list with **SEC Form ADV** as the index anchor. Every PE firm above \$150M AUM files one, so it's a real regulatory universe rather than names I could recall which attacks the 39% coverage problem directly.

After that, run it backwards: the index is already symmetric, so pasting a *fund's* URL and getting a list of acquisition targets is a change of query direction, not a new system. And I'd replace the backtest as the primary metric with blind professional rating of buyer lists against a baseline, because that measures the product rather than a proxy for it.